amU Faculté des sciences Aix Marseille Université

Master Physic and Applications

Numerical tools

Thierry ANGOT

2024-2025

Foreword

It is proposed here to acquire some knowledge on numerical methods by using PYTHON.

Numerical tools make it possible to simulate physical phenomena, especially those for which analytical resolution is difficult or impossible. They also allow the processing of experimental or theoretical data.

The programming language PYTHON, is free of charge, and is in continuous development. Optionally, but which can be very useful, especially for program development or data processing, PYTHON allows command line interactivity. It also offers very powerful graphical representation tools.

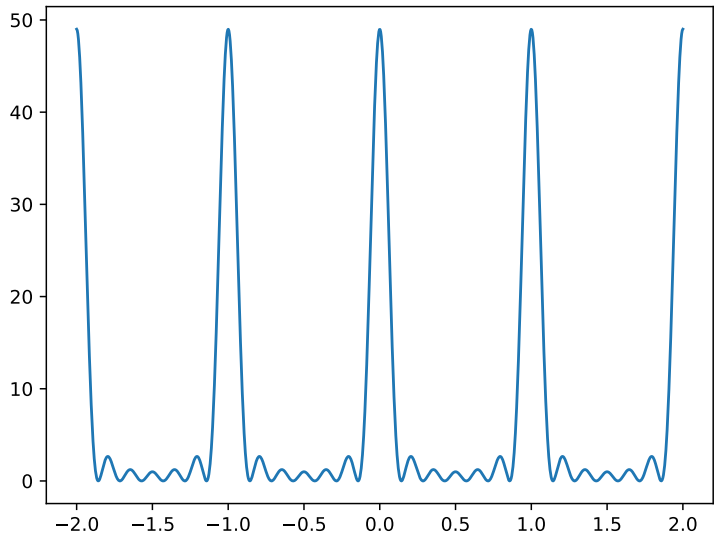


FIGURE 1 – $\frac{\sin^2(\pi(n+1)x)}{\sin^2(\pi x)}$

The software offered in this course is free of copyright and developed on the main digital platforms, using Mac OsX, Linux and Windows operating systems. Some elements allowing you to install and use them are described in the appendices.

Table des matières

1 Introduction **2**
- 1.1 Realising a figure . . . 2
- 1.2 Command line interaction . . . 4
- 1.3 Interaction using IPython . . . 4
- 1.4 Jupyter notebook . . . 5

2 Python basis **8**
- 2.1 Variables . . . 8
  - 2.1.1 Numerical variables . . . 8
  - 2.1.2 Variables séquences . . . 8
  - 2.1.3 while . . . 9
  - 2.1.4 for . . . 9
- 2.2 The conditions . . . 10
  - 2.2.1 if, elif, else . . . 10
- 2.3 Functions and procedures . . . 10
  - 2.3.1 Functions . . . 10
  - 2.3.2 Procedures . . . 11

3 Numerical tools **12**
- 3.1 Data reading . . . 12
- 3.2 Interpolation . . . 13
- 3.3 Random Numbers . . . 16
- 3.4 Integration . . . 17
- 3.5 Dérivation . . . 18
- 3.6 Ordinary differential equations . . . 19
- 3.7 Fourier transforms . . . 20

A Python Installation **23**
- A.1 Launching python : . . . 23
- A.2 IPython : Interactive Python . . . 24
- A.3 Jupyer notebook . . . 24
- A.4 Jupyter notebook and CoCalc . . . 24

B Librairies **25**
- B.1 **Matplotlib** . . . 25
- B.2 **Numpy** . . . 26
- B.3 **Scipy** . . . 26
- B.4 Other libraries . . . 27
- B.5 Softwares . . . 27

C Références **28**

Chapitre 1

Introduction

1.1 Realising a figure

To start this course about numerical tools for physics, we will approach the PYTHON language by an example : the command lines below are the ones used to realize the FIGURE 1 above (except its caption which was written in LaTeX).

Listing 1.1 – diffraction.py

```
# programme permettant de tracer l'intensite diffractee
# par un reseau periodique comportant 6 noeuds
"""
Created on Mon Feb 23 23:10:59 2015

@author: ThierryA
"""

import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-2., 2., 500)
n=6 #nombre d'objets diffractants
fig = plt.figure()
plt.plot(x, np.sin(np.pi*(n+1)*x)**2/np.sin(np.pi*x)**2)
plt.show()
```

The PYTHON language is a scripting language : a script consists of a series of command lines which will then be transformed by the PYTHON interpreter into machine code, one after the other, to be executed by the computer. It differs by this from languages (faster but not very interactive) requiring compilation and link editing before execution (FORTRAN or C language). Note that PYTHON was essentially written in C language (but be aware that there are versions of PYTHON written in other languages, and even a version of PYTHON written in PYTHON !

The script can be written with any text editor. It must be saved with a `.py` extension like the script above : `diffraction.py`.

We will see later that the script given as an example can be executed line by line.

The first character, #, indicates that the following text in the line is a comment. It is therefore not interpreted and is only used by the programmer to annotate his program to facilitate its reading at a later time or by other programmers. It is essential to comment on a program properly.

To comment several lines in a row without putting the crosshair character back at the beginning of each line, you can frame the comment lines with a series of """, [1]

They will therefore not be interpreted either, as is the case here for :

```
"""
Created on Mon Feb 23 23:10:59 2015

@author: ThierryA
"""
```

The program itself begins with
`import numpy as np`

i.e. by importing a library, the **numpy** library, to which we give here the name **np** to abbreviate its writing in the rest of the program. Indeed, PYTHON has a limited number of basic functions : https://docs.python.org/3.4/library/functions.html, but a large number of additional libraries have been developed, which allows to extend its capabilities considerably. **numpy** thus allows the management of arrays and matrices, sophisticated functions, code integration FORTRAN et C,... http://www.numpy.org.

PYTHON is an object-oriented programming language, and we will see later how to search some of the objects of the **numpy** library (or other libraries).

We then import a second library which will allow to realize the figure : **Matplotlib**. But rather than importing the whole library which contains many objects, http://matplotlib.org/api/index.html, we just import one of them : **pyplot**. We give it the name **pyplot** to shorten the typing because we will use some of the functions of this sub library, and this thanks to the command `import matplotlib.pyplot as plt`.

Grâce à **numpy** il est facile de créer pour la variable **x**, notre abscisse, une liste de 500 valeurs linéairement espacées, allant de -2. à 2, grâce à l'objet **linspace** et la commande
`x = np.linspace(-2., 2., 500)`.

The variable **n** is defined by the command line, and its physical utility is explained by the comment. `n=6 #number of diffracting objects`

A frame for the figure is created, to which the variable **fig** is assigned by calling the function **figure()** from the library `matplotlib.pyplot`. Note that the function is called without sending arguments as shown by the empty **()**. The command list is then :
`fig = plt.figure()`

From the list of data created for **x**, a two-dimensional graph is generated by calling the function **plot** of `matplotlib.pyplot`, the ordinate being computed by the function

$$\frac{\sin^2(\pi(n+1)x)}{\sin^2(\pi x)}$$

```
plt.plot(x, np.sin(np.pi*(n+1)*x)**2/np.sin(np.pi*x)**2)
```

The number π and the function *sinus* are not defined by default in PYTHON but declared in the **numpy** library. So they are called respectively by **np.pi** and **np.sin**.

1. We will see later that this comment mode is particularly appreciated to provide the basic instructions and parameters of the procedures which will then be available via the **help()** command described below.

And finally, we display the figure thanks to the function **show()** of `matplotlib.pyplot`, called here without argument : `plt.show()`

1.2 Command line interaction

The previous script is interpreted in one go, for example by executing in `Terminal`, `python nomduscriptpython.py` But it is also possible to enter it line by line, in a console, which can be displayed after entering the `python` command in **Terminal**, with the command line prompt that does not recognize the presence of `>>>`.

Exercice 1 : Enter the Listing 1.1 ? program in the command line, line by line, by pressing **enter** after each line entered. This will eliminate the need to enter the comment lines. If you have entered your script correctly, you should see a window open with the figure.

Commande help()

You can get detailed information from any module by using the **help()** function. For example, if you want to know more details about the sine function (from the numpy library) used previously, you can enter : `>>>help(np.sin)`

or

`>>>help(numpy.sin)`

For a basic function of PYTHON, for example the absolute part, you simply enter :

`>>>help(abs)`

1.3 Interaction using IPYTHON

You can also interact with a modified version of PYTHON, which is called IPYTHON, the 'I' indicating interactivity. The interface is then in many ways comparable to that of `Matlab`®.

From `Terminal` we call IPYTHON by the command :

`ipython`

which brings up a command prompt of the following form :

`In [1] :`

If one performs a simple calculation, for example,

`In [1] : 3/7`

The answer appears in the form

`Out [1] : 0.42857142857142855`

[2]

`In [2] :`

`In [2] : 3./7.`

It is a good habit to write real numbers with their comma (i.e. a period in English) which avoids important calculation errors thanks to a better compatibility with previous versions of PYTHON, or other languages that respect this rule (FORTRAN for example).

IPYTHON offers a number of additional possibilities compared to PYTHON which will be detailed to you as we go along. You can have an overview of these features by typing : `In[3]:?`

-First of all, it simplifies the syntax to make it lighter to use.

2. The result would be different with PYTHON2 because the calculation would have considered two integers, and would therefore give `0` It is therefore necessary to specify that the number is real, which is done by writing to the numerator `'3.'` (which is the condensed form of `'3.0'`) instead of `'3'`.

-Then IPYTHON keeps the history of inputs and outputs which allows to easily find the past commands.

The complete history of the session can be obtained with the special command : `%history`.

Recalling a previous command is very easy since `In` and `Out` are numbered. For example, you can enter : `In[4]:Out[1]+6`

to obtain the result of the `3/7+6`

-It offers special commands, 'magic', which provide additional functionality. Besides you can get information about these commands by typing `In[5]:%magic`

One of the most useful commands is **%psearch a*** which lists all modules starting with the letter 'a' (of course it works with any letter), and this command can be used in the simplified form **?a*** as well. We will describe other 'magic' special commands as the needs of this course arise.

- The command `help()` previously described is simplified in IPYTHON by **?**. For example to get information about the whole part function, we will write : `In[6]:?int`

or also

`In[6]:int?`

Exercice 2 : Repete exercice 1 in IPYTHON interface .

At the end of this exercise, you can experiment with another useful command : if you want to save the commands `In[1]` to `In[5]`, you can use `save mysavedhistory 1-5`. You can then reload all these commands with `%load my`, or execute them directly with `%run mysavedhistory`.

Exercice 3 : Find all the objects available under IPYTHON with the command `?*`, and get the documentation on these objects.

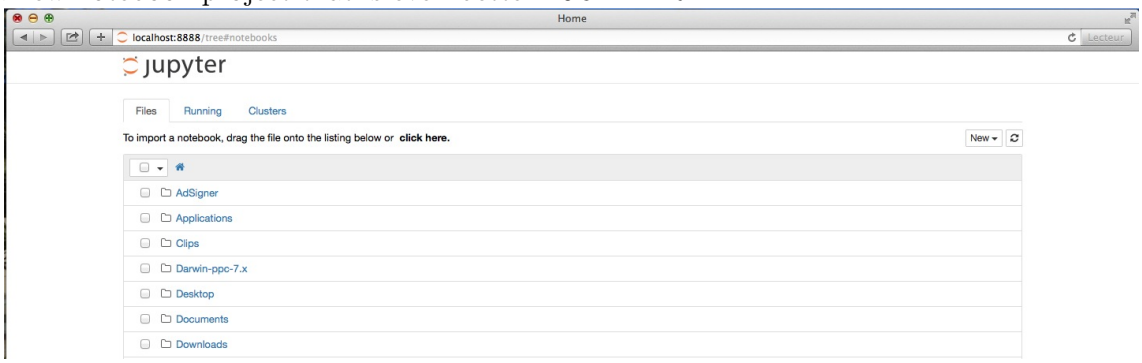
1.4 JUPYTER notebook

Another way to interact with PYTHON is to enter the following command in `Terminal`

`jupyter notebook`

This is the recommended mode of use for the continuation of this course, as it presents a very pleasant interface (in the manner of Mathematica®) as you will soon realize. The python interface then appears in your favorite web browser.

It is an evolution of a former project called IPYTHON. notebook. And there is a beta version of a new notebook project that is even better : JUPYTERLAB.



The `Files` menu at the top left allows you to navigate through your folders. It is advisable to create a folder where all the notebooks you will develop will be stored. This can be done by using the tab on the right side of the page and selecting the menu item "Verb¿New" and then selecting the menu item "Verb¿Folder".

By ticking the button on the left of your new folder **untitled folder** you will see at the top right the possibility to rename it (for example to myNotebooks) or to delete it.

Then navigate in this folder by double-clicking on it, and create your first notebook thanks to the menu `New` and the submenu `Python3`. Then, at the top of the window near the JUPYTER logo, by clicking on `Untitled` you can rename your first notebook. Choose an explicit name, linked to the program you are going to develop, so that you can find your way around later.

You then have a code entry field as below.

```
In[ ]:
```

!!! Note that you can now enter several lines of code, the **Enter** or **Return** key (return to line) to go to the next line.

Important

You can execute all lines of code by executing the keystroke sequence **Shift-Enter** (Shift-Enter) simultaneously.

Exemple

```
In[1]: a=1
       b=2
       print(a/b)
```

```
0.5
```

The result is displayed immediately (using the `print()` function) below the lines of code, and a new command line prompt appears.

You can now change the values of a and b directly in the line you typed and restart the calculation by executing the sequence **Shift-Enter**. The new result is displayed, and the history is incremented :

```
In[2]:
```

Important

From now on, you can enter and execute the following line for any use of your notebooks.

```
In[1]: %pylab inline
```

```
Populating the interactive namespace from numpy and matplotlib
```

This allows loading the **Pylab** library, which in turn adequately loads (by resolving possible problems of module name conflicts) the **Numpy** and **Matplotlib** libraries that provide an environment already well adapted to scientific computation. The `inline` command allows to obtain the figures directly inserted in the notebook.

If you wish to have an overview of all the modules imported by (Pylab), just use the `dir` command as follows : `dir(pylab)`

Note finally that you can use the fields, not only to code in PYTHON language, but also to enter text or comments, normally, or in markdown language (HTML or markdown), or also to write equations in LaTeX. To do this you must select Markdown in the submenu of JUPYTER instead of code.

Chapitre 2

Python basis

2.1 Variables

There are many variables[1]. in PYTHON. Depending on their type, PYTHON will manage their storage in memory differently. There are two categories : numeric variables and sequences.

2.1.1 Numerical variables

There are three types of numerical variables : integers, real numbers and complex numbers. Integers (`int`) : The simplest are limited to $2^{31} - 1 = 2147483647$. They are recognized as such by PYTHON when there is no point. Longer integers, will be considered as long integers and managed whatever their size automatically with only one limit on the computer memory.

Real numbers (`float`) : these are numbers with a decimal point, for example 3.1415927 (in French we would use the decimal point but it is the point which is retained in all programming languages and as it is probably the case for your desktop calculator). They require more memory and make calculations slower than integers.

Complex numbers : they use $j = \sqrt{-1}$ and for example defining a number like $z1 = 2.2+1.7j$ is automatically handled by PYTHON which will be able to perform all arithmetic operations (+,-,*,/) correctly between complex numbers. The real part of the complex number can be obtained by the function `real()` and the imaginary part by `imag()`.

For example : `z1.real()` will give `2.2` and `z1.imag()` will give `1.7`

The conjugate complex is obtained by `z1.conjugate()`.

After execution of the command `%pylab inline`, the real and imaginary parts as well as the conjugated complexes are respectively obtained by `real(z1)`, `imag(z1)`, and `conjugate(z1)`.

It is then possible to apply other functions to complex numbers, such as trigonometric functions.

For example, `sin(z1)` will provide `2.2866828765713958-1.5569573489096324j`

2.1.2 Variables séquences

Sequences are made up of different objects, which can be numbers or typographical characters, or other types of objects, and are associated with a variable name. The objects of the sequence are separated by commas, and referenced by an index in square brackets, after the sequence name (i.e. the variable associated with the sequence).

For example we can define the following sequence :

`leptons=("electrons", "muons", "tauons")`,

and access the individual items in this list by using `leptons[0]` to get `electrons`, `leptons[1]` to get `muons` and `leptons[2]? to get \verb`tauons¿. 0.5cm

1. In object-oriented programming, the variables contained within the objects are called attributes

There are different categories of sequences, presented below, starting with those that may be the most useful to us :

Of course the lists can be more elaborate as : `matr=[[2.,4.],[8.,16.]]` and accessing the first element of the list by `matr[0]` which gives `[2.,4.]` or the first term of the second element of the list by `matr[1][0]` which gives `[8.]`.

Tuples : are elaborated from simple parentheses. Contrary to lists, you cannot modify the elements, but tuples are of course less resource intensive (than lists).
Here is what it looks like for example : `grec=('alpha', 'beta',delta',gamma')`.

Dictionaries : they can be recognized by the braces that enclose the data structure.
For example, you can create a dictionary of constants to be used in a calculation : `const={'cm-1':8.066}`. To use the alpha constant in a calculation, we will write : `360*const['cm-1']` which allows here to convert the frequency of a stretching mode of a C-H vibration observed at 360 meV, into cm^{-1}. `const.keys()`
`const.values()`
`const.items()`
which return objects called views.
sectionThe loops There are two types of loops, the "while" loop and the "for" loop.

2.1.3 while

footnoteThe above `while` command actually combines loop and condition. To execute an instruction in a loop until a condition is met, for example to get all even numbers up to and including ten, write `i=2`

```
while(i<=10):
   print(i)
   i=i+2
```

Important

Note the `':'` at the end of the instruction line `while`, and the indentation (tab or 4 spaces) for instructions within that loop. The absence of `':'`, or a wrong indentation will cause an error in the execution of your code.

2.1.4 for

In the same way, the same result can be obtained with the function `for` in the following way :

```
for i in range(2,12,2):
   print(i)
```

We notice the presence of the `':'`, and of the indentation, as previously, for the loop `for`. One also notes the use of list.

`range(2,12,2)` indicates successively the start number, the finish number and the step. The loop stops when it reaches the end number of the array (here `12`) but without executing. `range()` is a basic function of PYTHON. Note that a list of these numbers can be created by integrating loop and condition in the following way :
`[i for i in range(0,12) if i\%2==0]` The sign % is the operator `modulo`.

In a similar way, thanks to the `numpy` library imported by `%pylab`, we can create a loop on real numbers, by the function `arange()` :

```
for i in arange(3.,4.,0.1):
print(i)
```

But when you need a very good precision, you can use the instruction "verb¿space".

```
for i in linspace(3.,4.,15):
print(i)
```

`space(3.,4.,15)` indicates successively, the starting and final number, and the total number of points between these two terminals.

2.2 The conditions

One will notice the presence of the `':'`, and the indentation, as before.

2.2.1 if, elif, else

In order to check whether a condition is true or false, the command `if` is used.

```
if x==2:
   print('x is equal to 2')
```

in case this condition is not verified, the command `else` is used.

```
if x==2:
   print('x is indeed equal to 2')
else:
print('x is not equal to 2')
```

and it is possible to test other conditions with the `elif` command.

```
if x==2:
   print('x is well equal to 2')
elif x<2:
   print('x is smaller than 2')
elif x>2:
print('x is greater than 2')
```

2.3 Functions and procedures

Functions and procedures [2] share the same `def` command. They are not formally differentiated in PYTHON.

2.3.1 Functions

A function takes one or more parameters and usually returns a single value.

For example, to obtain the distance between two coordinate points, we will write a function as follows :

```
def distance(x0,y0,x1,y1):
   return sqrt((x1-x0)**2+(y1-y0)**2)
```

The distance between two points can then be known at any time by calling this function. For example : `distance(1,2,3,4)` will provide the value `2.8284271247461903`. [3]

2. In object-oriented programming, procedures contained within objects are called methods.

3. But be aware that PYTHON allows the creation of anonymous functions using a constructor called `lambda` which we will not present in this lesson.

2.3.2 Procedures

The procedures are no different from the functions. They are sometimes distinguished from functions by the absence of the verb "return" instruction and the fact that they may return nothing. For example, the following procedure prints the value of the square root of a number, if it exists : [4].

```
def testracine(nb):       """
   testracine(number) -> print a number or a character string
      Print the square root of a real number
      or a string indicating it does not exist if the number is negative
   """
   if nb<0
      print('il n'y a pas de racine réelle pour ce nombre')
   else:
      print('la racine du nombre est ',sqrt(nb))
```

but does not return any value. You can test this procedure with `testracin(2)` and `testracin(-3)`, for example.

Note that variables in procedures and functions are local (dummy variables).

4. Note here the use of comments to describe the behavior of the procedure. You can then access this descriptive information using **help(testracine)**

Chapitre 3

Numerical tools

We will discuss here a number of numerical tools for physics.

3.1 Data reading

The function `loadtxt()` from the `numpy` library is very useful for loading data from a text file. The following example shows how to read data. The data is an infrared spectrum of ethanol, which you can find in JCAMP-DX format on the NIST website. (http://webbook.nist.gov/cgi/cbook.cgi?ID=C64175&Units=SI&Type=IR-SPEC&Index=0#IR-SPEC). This file can be read with a simple text editor. Note that the first 25 columns give information about the following data and may not be loaded, thanks to the option `skiprows=25`. For more information about the JCAMP-DX format you can refer to the following article :http://www.jcamp-dx.org/protocols/dxir01.pdf. The infra-red data consists of 11 columns. The first one (to which the variable x is assigned) represents the wave numbers expressed in cm^{-1}. The next 10 columns represent the intensities for x successively incremented by 4 cm^{-1} with respect to the value of the first column. The option `unpack=True` allows a transposition of the data to facilitate their reading.

The nested loops allow to generate the set of wave numbers on the abscissa (x list) and to reorder the intensities (y list) accordingly.

```
In [1]: %pylab inline

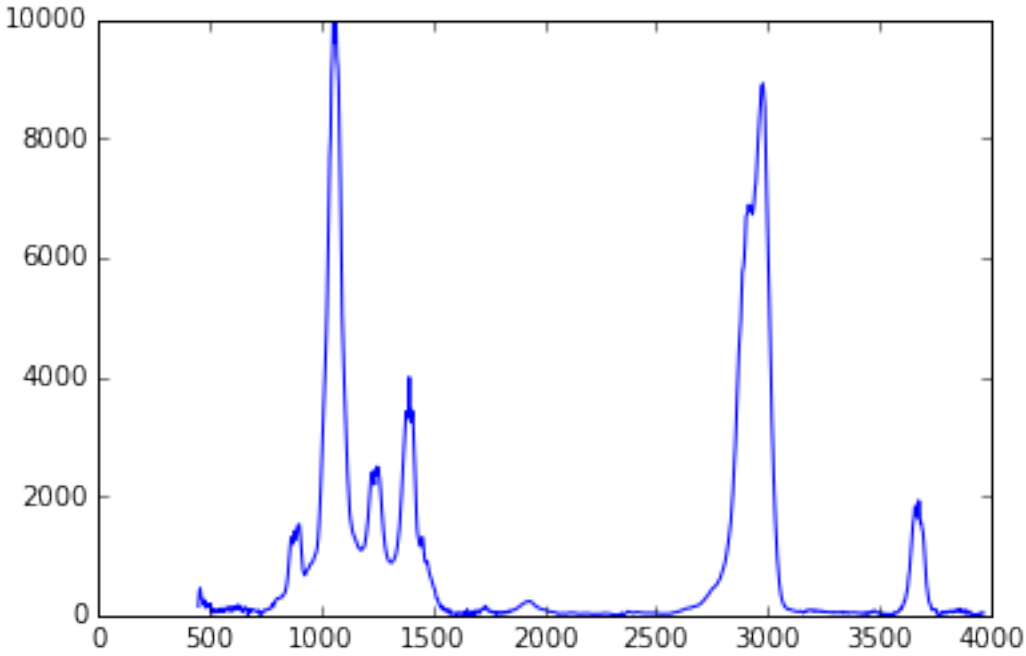
Populating the interactive namespace from numpy and matplotlib

In [2]: data=loadtxt('64-17-5-IR.jdx',unpack=True, skiprows=25)

In [3]: x=[];y=[]
        for i in range(0,len(data[0])):
            x.append(data[0][i])
            for j in range(1,10):
                x.append(data[0][i]+4*j)
            for j in range(1,11):
                y.append(data[j][i])

In [4]: plot(x,y)

Out[4]: [<matplotlib.lines.Line2D at 0x10582b240>]
```



The previous graph represents the values of y as a function of x. The different vibration modes of the molecule observed in the infrared are represented.

3.2 Interpolation

Experimental data often require interpolation. For example, to produce a smoother curve that masks variations related to the measurement statistic, or a fit with a well-defined function to extract relevant physical parameters. The spectrum obtained previously will be used to illustrate this section. We will fit a part of this spectrum with a Gaussian curve :

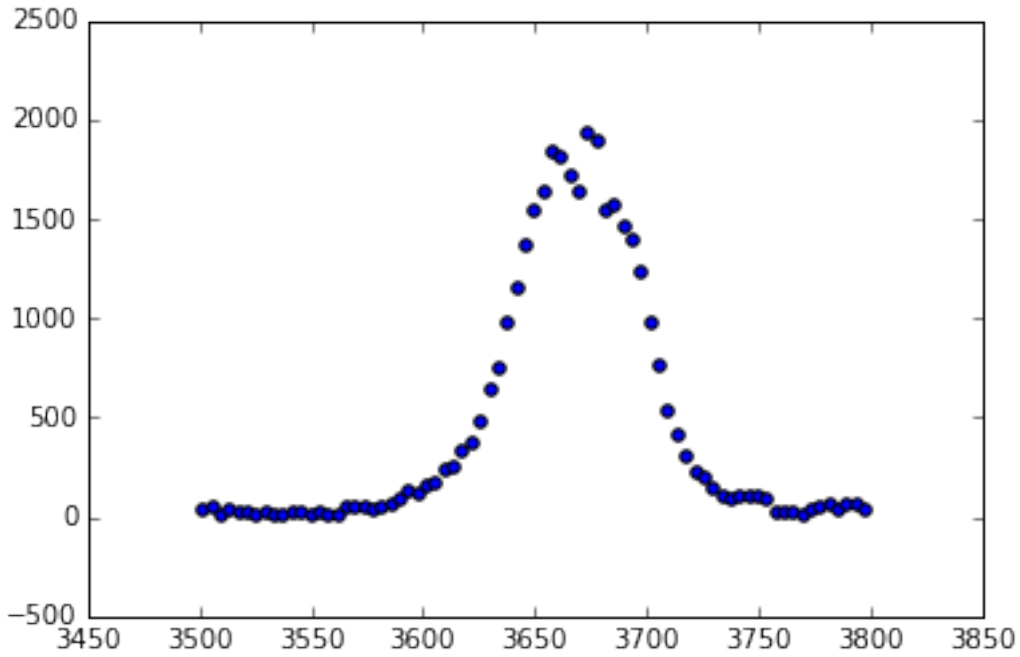
$$\frac{1}{\sigma\sqrt{2\pi}} \exp \frac{-(x-x_0)^2}{2\sigma^2}$$

The commands below will be entered following those in the previous section.

```
In [5]: #procedure pour determiner la valeur de l'indice correspondant
        #au premier x superieur ou egal a une valeur proposee
        def trouveindex(valeur,x):
            for index, nombre in enumerate(x):
                if(nombre>=valeur):
                    break
            return index
```

```
In [6]: #on cherche les indices de la zone sur laquelle on va travailler
        i1=trouveindex(3500,x)
        i2=trouveindex(3800,x)
        #on cree de nouvelles donnees pour cette region d'interet
        xp=x[i1:i2]
        yp=y[i1:i2]
        scatter(xp,yp)
```

Out[6]: <matplotlib.collections.PathCollection at 0x105958668>



In [7]:
```
#fonction definissant la gaussienne
def gaussian(x, a,x0,sigma):
    return a*numpy.exp(-(x-x0)**2/(2.*sigma**2))
```

In [8]:
```
imax=max(yp)#determine les coordonnees du maximum d'intensite
m=trouveindex(imax,yp)

print('indice=',m,'; coordonnees=(',xp[m],',',yp[m],')')

init=[yp[m],xp[m],1]#valeurs approximatives initiales pour l'ajustement
```

```
indice= 43 ; coordonnees=( 3674.0 , 1926.0 )
```

In [9]:
```
#curve_fit va permettre d'optimiser les parametres de la gaussienne
from scipy.optimize import curve_fit

bestval,covar=curve_fit(gaussian, xp, yp, p0=init)

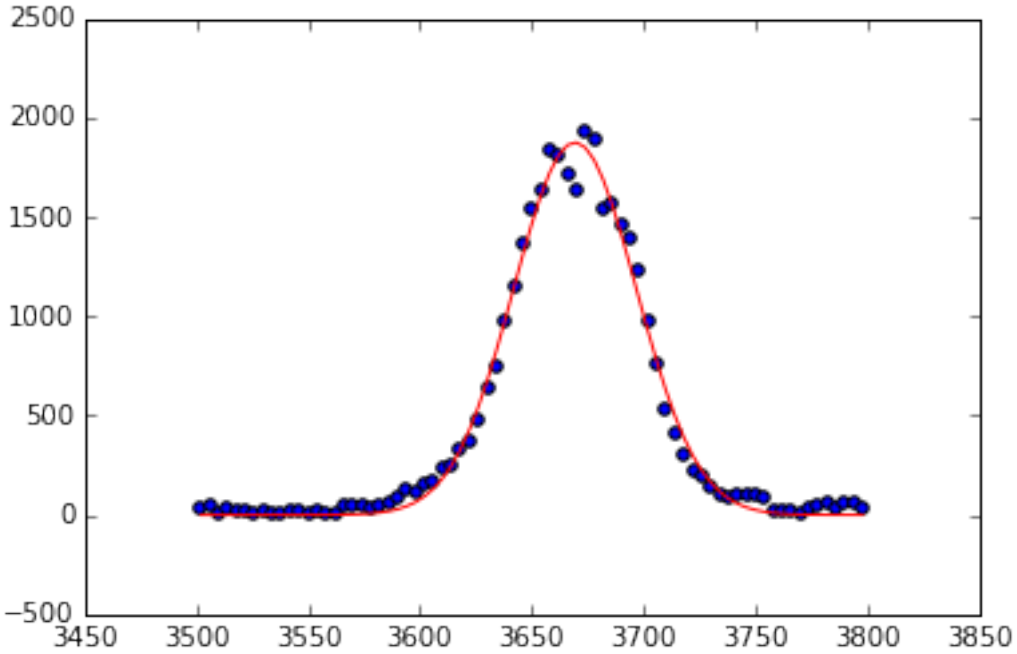
print('Intensite=',bestval[0],'Position=',bestval[1])
print('Largeur a mi-hauteur:FWHM=',2*sqrt(log1p(2))*bestval[2])
```

```
Intensite= 1874.62329658 Position= 3669.3603846
Largeur a mi-hauteur:FWHM= 57.5968317091
```

In [10]:
```
#on cree un liste a partir des resultats de l'ajustement
 #gaussien des donnees
 y_optim=gaussian(xp,bestval[0],bestval[1],bestval[2])
```

```
#superposition des resultats experimentaux et de l'interpolation
scatter(xp,yp)
plot(xp,y_optim,c='red')
```

```
Out[10]: [<matplotlib.lines.Line2D at 0x105818e80>]
```



We limit ourselves to the analysis of the infra-red absorption peak of the largest number of waves. To begin, we define in `In [5]` : a `trouveindex` procedure that will allow us to determine the index for a desired value.

`In[6]` : look for these indices for the values 3500 and 3800 which will frame our area of interest. We create two new data lists for this region that we display as a dot plot.

`In [7]` : the Gaussian function is defined.

`In [8]` : to facilitate the adjustment of the data, we determine the maximum of the peak, its index in the list, which allows us to print the coordinates. These will be used as input parameter for the adjustment, inserted in a new list, `init`. The last parameter `sigma` related to the width is set equal to 1.

`In [9]`: To perform the fit, we need to import the procedure `curve_fit` from the library `scipy.optimize`. We give it as parameter the function needed for the fit, the stitch-lists, and p0=`init`, the list of initial parameters for our Gaussian. The function returns two lists which have given the names `bestval` and `covar`. `bestval` contains the values of the parameters of the Gaussian after optimization, and `covar` contains statistical parameters that could be used to check for example the quality of the optimization.

`In [10]` :. An intercept list `y_optimal` using the optimized parameters and the Gaussian function is created for each value `xp` to generate the fitting function.

Finally this fitting function is plotted in superposition with the infra-red data.

Exercice 4 : Repeat the previous analysis using a Lorentzian function, which you will define. Remember that the Lorentzian function is expressed mathematically :

$$\frac{\frac{2}{\pi\Gamma}}{1+(\frac{x-x_0}{\Gamma/2})^2}$$

Compare the fit with the Gaussian. From a physical point of view, which function should best reproduce the data ?

Exercice 5 : Adjust simultaneously, using the appropriate function, the two strongest peaks between 2500 and 3250 cm^{-1}. Compare the mid-height width of these peaks, as well as with the width obtained in the previous example.

3.3 Random Numbers

Random numbers are important in many problems.
For example, consider the following integral :

$$\int \exp(-at^2)$$

Here we will take $a = 1s^{-2}$. If the integral here can be formally determined over the interval $[0.\infty]$ (the value is $\frac{\sqrt{\pi}}{2}$), its calculation is impossible over a finite interval, for example [0.1]. There are many numerical methods to perform this integration, we will use here the surface method : the principle of this method is to perform a random draw, and to count the number of points below the curve we want to integrate that satisfy the condition y <= fn(t). This then allows us to estimate the integral.

```
In [1]: %pylab inline
```

```
Populating the interactive namespace from numpy and matplotlib
```

```
In [2]: def fn(t):
            return exp(-t*t)
```

```
In [3]: seed(None)
        for i in range(1,9):
            n=10**i
            t=zeros(n);y=zeros(n);res=zeros(n)
            t=rand(n)
            y=fn(t)
            u=rand(n)
            res=[1 for i,j in zip(u,y) if i<=j]
            print('I=',asarray(res).sum()/n,', pour n=',n)
```

```
I= 1.0 , pour n= 10
I= 0.73 , pour n= 100
I= 0.749 , pour n= 1000
I= 0.7524 , pour n= 10000
I= 0.74525 , pour n= 100000
I= 0.747024 , pour n= 1000000
I= 0.7467478 , pour n= 10000000
I= 0.74681356 , pour n= 100000000
```

We define the function to be integrated by the two command lines of `In [2]:`

On the first line of `In[3]:`, the function `seed()` allows to initialize the seed allowing the generation of the random numbers (in fact, we are dealing with 'pseudo' random numbers). The argument, `none` or no argument ensures that the seed will always be different, currently by initializing the seed using the system clock. In this way it becomes impossible to know in advance the uniform distribution of random numbers that will be generated using the function `rand()`.

The loop on `i` will allow us to generate an increasing number of random numbers (from 10 to 100,000,000), to study the impact of the number of draws on the precision of the calculation.

Three list tables `t`, `y` and `res` (in this case one-dimensional) are initialized with the instruction `zeros()`. `t=rand(n)` fills the `t` array with random numbers, and the corresponding values of the function are stored in `y`. A second random number array is generated with `u=rand(n)`.

The next line is a bit complicated : `res=[1 for i,j in zip(u,y) if i<=j]` : It allows to combine the two lists `u` and `y` into one thanks to the instruction `zip(u,y)`, to read each of the elements of the pairs created at each index thanks to `for i,j in`, to compare these elements using `if i<j`, and to return the value 1 in the case where `i<=j` in the tuple `res`.

Exercice 6 : Similarly, calculate the integral of the function $\sqrt{1-x^2}$ in the interval [0,1] and compare the results with the theoretical value.

The use of random numbers as an integration method is not very efficient. You may have noticed a noticeable slowdown in the calculation of the integral for high n values. However, this method can sometimes be indispensable, especially when the number of variables is high.

3.4 Integration

As a general rule, for integration, the use of the `integrated-module of the \verbscipy¿` library is preferred. In particular, the previous calculation can be very efficiently performed using the instruction `integrate.quad(fn,0,1)`, to which the function and its integration bounds are provided as arguments. The result is instantaneous and obtained with a known precision.

```
In [1]: from math import exp

In [2]: def fn(t):
            return exp(-t*t)

In [3]: import scipy
        from scipy import integrate
        result=integrate.quad(fn,0,1)
        print(result)

(0.7468241328124271, 8.291413475940725e-15)

In [4]: res=result[0]
        print(res)

0.7468241328124271
```

The result is a list from which we can of course choose the element, here `result[0]` for the value of the integral.

Exercice 7 : Calculate the integral of the function $\sqrt{1-x^2}$ in the interval [0,1] and compare the results with the value of π.

3.5 Dérivation

There is also a function to numerically calculate the derivative of a numerical function. It can be found in the library `scipy` in the module `misc`. As an example, we calculate the second derivative of the function $exp(-at^2)$ with $a = 1s^{-2}$.

```
In [1]: %pylab inline
        from scipy import misc

Populating the interactive namespace from numpy and matplotlib

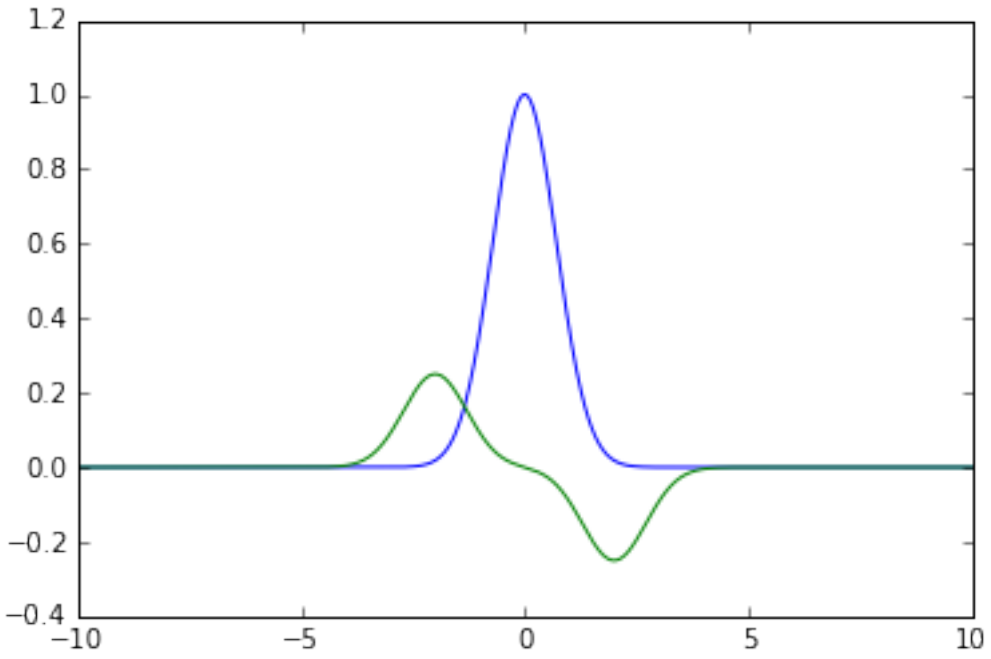
In [2]: def fn(t):
            return exp(-t*t)

In [3]: t=arange(-10,10,.01)

In [4]: d2fn=misc.derivative(fn,t,2)

In [5]: plot(t,fn(t),t,d2fn)

Out[5]: [<matplotlib.lines.Line2D at 0x105df3048>,
         <matplotlib.lines.Line2D at 0x105df3390>]
```



The procedure `derivative()` takes the function and the variable as arguments by default. The derivative will then be of order 1 and can be completed with the order of derivation as an

additional argument as here to obtain the second derivative (2).

Exercice 8 : Calculate the derivative of the function $sin(x)$ in the interval [0,10] and compare the results with the function $cos(x)$.

3.6 Ordinary differential equations

Integrating a first-order differential equation is easily done with PYTHON thanks to the `scipy` library and the modules of `scipy.integrate` We will use here the `odeint` procedure. [1]

We will start by importing this procedure, thanks to the command

`from scipy.integrate import odeint`

For illustration purposes, we will calculate here the radioactive decay of Tritium, the isotope of Hydrogen, which is involved in the thermonuclear fusion reaction to be carried out in ITER. Its half-life (τ) is 12.32 years.

The differential equation expressing the decreasing with time is written : $\frac{dN}{dt} = -\frac{t}{\tau}$. and so we manufacture this function. For `N`, the `odeint` procedure requires sending a list for the initial conditions and needs to return a list for `N? hence the \verbN[0]¿` and the `[dN]`. For more information and a better understanding of the rest of the program, consult the documentation by running `odeint?`. Then we define a time base of 100 years by taking 25 points in this interval.

`t=linspace(0,100,25)`

`res=odeint(deriv,n[0],t)` allows to digitize point by point, all $\frac{100}{25}eme$ radioactive decay. The rest of the code allows to visualize the results with the help of a graph. The integration can be performed formally $exp-\frac{t}{\tau}$ and is superimposed on the numerical data. Finally, note the use of LaTeXfor caption formatting.

```
In [1]: %pylab inline
        from scipy.integrate import odeint

Populating the interactive namespace from numpy and matplotlib

In [2]: tau = 12.32/log(2.)

        def deriv(N, t):
            dN=-N[0]/tau
            return [dN]

        start = 0
        end = 100
        steps = 25
        t = linspace(start,end,steps)

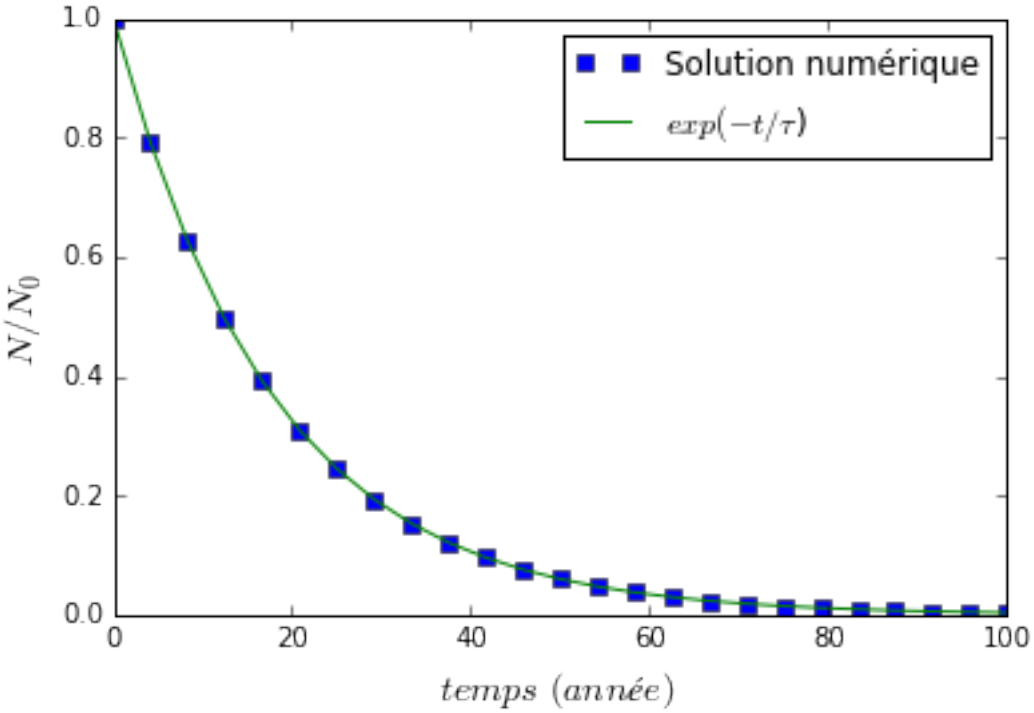
        N0=[1]
        res=odeint(deriv,N0,t)

        plot(t, res, 's',   label="Solution numérique")
        plot(t, exp(-t/tau), label="$exp(-t/\\tau$)")

        xlabel("$temps\ (année)$ ",  fontsize=14)
        ylabel("$N/N_{0}$", fontsize=14)
```

1. For new codes, it is advised to replace this procedure by `scipy.integrate.solve_ivp` .

```
legend()
show()
```



Exercise 9 : . We consider two harmonic oscillators of the same mass and stiffness constant. One of the oscillators is moved from its initial position, without giving an initial velocity. Write a function calculating the velocity and acceleration of the two masses (the initial conditions will have been stored in a table) as a function of position, velocity and time. After integration, plot the variation of the position of the oscillators. Plot the phase potrait.

3.7 Fourier transforms

Let's generate a triangle function for which we will calculate the Fourier transform. To do this we will use the collections of `signal` objects from the `scipy` library.

```
In [1]: %pylab inline
        from scipy import signal
```

```
Populating the interactive namespace from numpy and matplotlib
```

```
In [2]: f=1. #frequence du signal
        t = np.linspace(0, 1, 500, endpoint=False)
        T=1./f
```

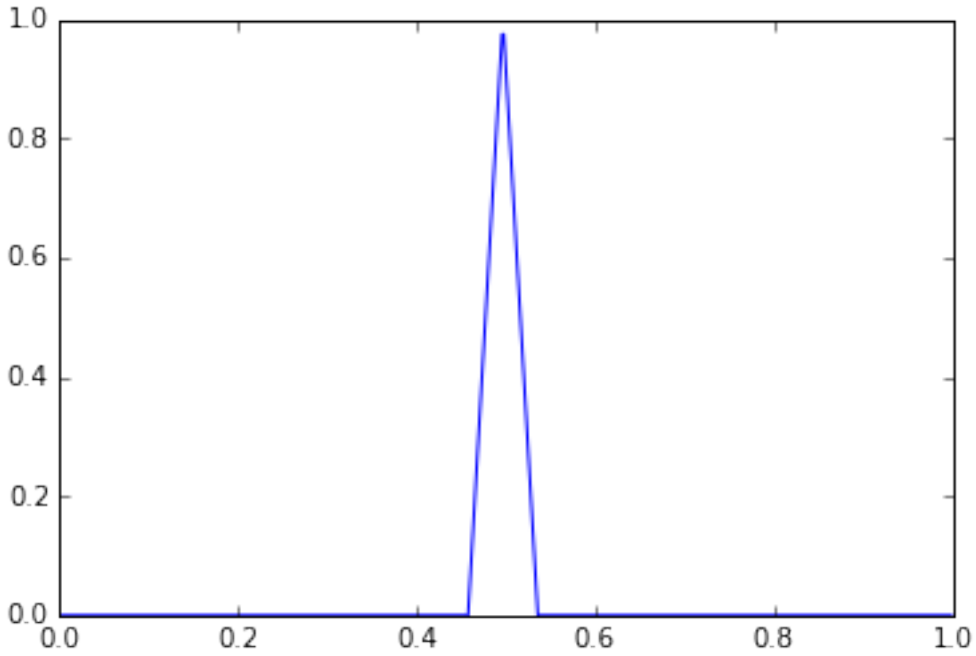
```
In [3]: w=40 #largeur de la fonction
        sig=zeros(500-w)
        ttemp=linspace(0,1,w)/500
        stemp=(signal.sawtooth(2 * pi * ttemp/T*500,[.5])+1)/2
```

```
In [4]: sig=insert(sig,250-(w/2)-1,stemp)
        size(sig),size(t) #taille des listes

Out[4]: (500, 500)

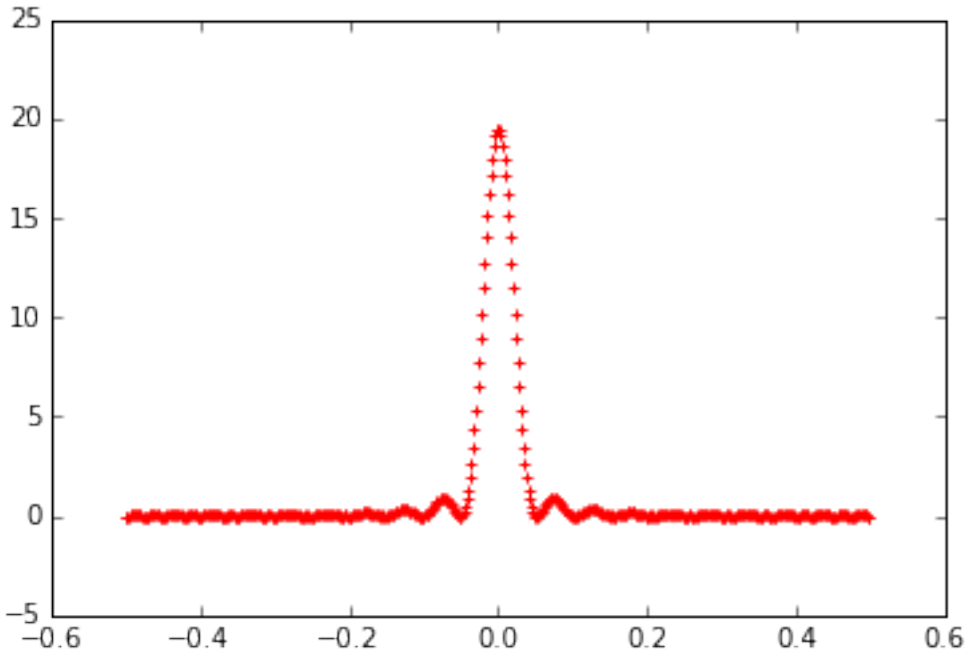
In [5]: plot(t,sig)

Out[5]: [<matplotlib.lines.Line2D at 0x106eb9c18>]
```




```
In [6]: y=fft.fft(sig) #FFT
        ff=fftfreq(size(t))
        scatter(ff,abs(y),s=10,marker='+',c='red')

Out[6]: <matplotlib.collections.PathCollection at 0x10707f4e0>
```



Exercice 10 : Generate a cardinal sinus between -1 and 1 with 500 points. Perform an inverse Fourier transform. What is the function obtained ?

Annexe A

PYTHON Installation

PYTHON can be installed directly from https ://www.python.org/downloads/

Note that it is already installed by default on your Mac OsX and Linux operating system, but under Windows its installation is required.

There are two development versions PYTHON (version 2 and version 3). We will use here version 3 which only continues to evolve, version 2 being updated only to solve security issues.

Generate a cardinal sinus between -1 and 1 with 500 points. Perform an inverse Fourier transform. What is the function obtained ? For scientific work, there is a fairly complete suite called Anaconda which is highly recommended for learning this course. It can be downloaded for free at https ://www.anaconda.com Install the latest version 3.x

Anaconda installs PYTHON and many scientific libraries allowing data analysis and scientific computation. Among these, we will use **Matplotlib** for graphical representation, and **numpy** a scientific library useful in particular for the management of tables and matrices.

To get a complete list of the libraries already installed, you can enter the **Terminal** command in the application **Terminal** to get the `conda list`

If you want to know what other libraries are available, you can enter the command `conda list` To install a new library, just execute the command `conda install library_name`

You can use the `'Launcher'` does not provide the installation to launch PYTHON and in particular IPYTHON notebook described below. Nevertheless I propose an alternative in the following sections.

A.1 Launching python :

Mac OsX et Linux launch the **Terminal** window and write `python`.

Windows : launch the **cmd.exe** window to use `MS DOS` . in the command prompt, write `python.exe`.

In both cases you should end up with the command prompt of PYTHON.

```
>>>
```

This is the default mode. If you have generated a program PYTHON in a text editor `myprogram.py`, you can execute it directly without entering PYTHON, by executing in the console `python myprogram.py` (you must of course locate the directory where you have your source file, or indicate the path of this file).

A.2 IPYTHON : Interactive PYTHON

To work in interactive python, the use of IPYTHON is recommended : in **Terminal** or the Windows command prompt, enter the following command (sans le $ bien sur) :

```
$ ipython --pylab
```

or

```
$ ipython
```

followed by the magic command at the command prompt d'IPYTHON

```
In [1]: %pylab
```

By importing PyLab, we find ourselves with a Mathematica® type interface, and most of the libraries needed for scientific computation and data analysis. To visualize the objects to which you have access, enter

```
In [2]: pylab?
```

A.3 JUPYER notebook

After installing Anaconda, go to **Terminal** or the Windows command prompt, and enter the following command :

```
jupyter notebook
```

It is possible that the version of PYTHON2 has been installed, depending on your initial installation with **Anaconda**. If you want to use a different version, for example PYTHON3, then you must enter the following commands, which allow you to create a specific environment (you can use this method if you want to install several versions of PYTHON) :

```
$ conda create -n py3k python=3 anaconda
```

which allows you to install version 3. Then,

```
$ source activate py3k
```

whenever you want to use this environment. Then you enter the command

```
$ jupyter notebook
```

C'est actuellement cette version que je vous propose d'utiliser pour étudier ce cours.

Finally, with a little more interactivity, there is a beta version of a notebook called JUPYTER LAB that you can try to install and test while waiting for the stable version 1.0.

A.4 JUPYTER notebook and CoCalc

If you have difficulties installing the Anaconda suite (or you don't want to), I suggest you use an online version of JUPYTER notebook : CoCalc. It is a collaborative computing platform that allows you, among other things, to perform calculations under PYTHON and to share your work.

Open your favorite browser (Firefow for example) and browse the page :

https ://cocalc.com

Then proceed as follows : after creating an account, generate a new project that you select. In the new view, choose the 'New' tab and select for file type : IUPYTER notebook. Then choose the kernel PYTHON3. You can now start coding !

Annexe B

Librairies

The number of libraries available is very high, and the use of each one requires a great deal of learning, as you can see from reading this course. textbfMatplotlib, textbfNumpy and textbfScipy are the most essential for scientific computation.

B.1 Matplotlib

Library for the realization of 2D graphs : http ://http ://matplotlib.org/

To add to the examples of this course, I propose the following objects :

Humorous graph

To draw a slightly distorted graph, giving it the look of a cartoon, you can use the following function : After importing the usual libraries :

`%pylab inline`

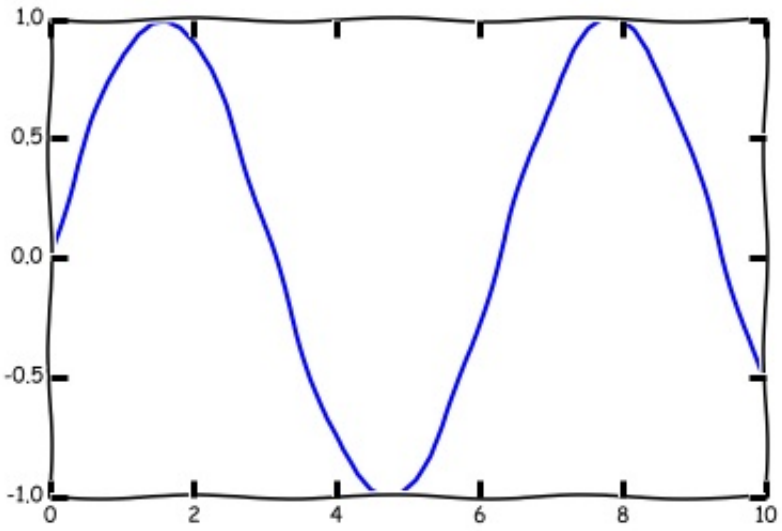
The function `xkcd()` is executed.

And if, for example, we draw a function, arbitrarily defined below : `x=linspace(0,10,50)`

`y=sin(x)`

After executing the function `plot`

`plot(x,y)` the graph has a deformed look like a comic book.



3D graph

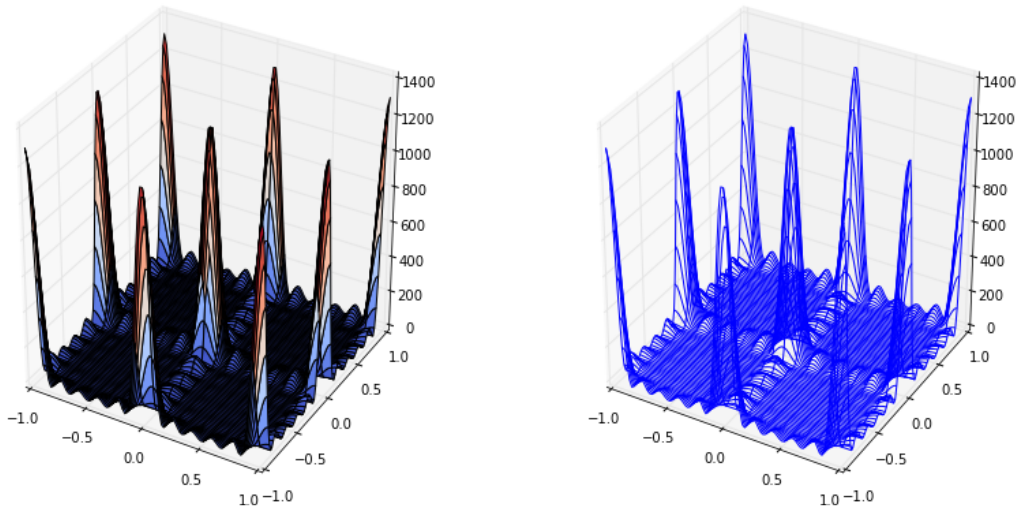
Matplotlib allows the visualization of 3D data using `mplot3d` from the `toolkits` library as in the following example :

```
In [1]: %pylab inline
        from mpl_toolkits.mplot3d.axes3d import Axes3D

Populating the interactive namespace from numpy and matplotlib

In [2]: fig = figure(figsize=(14,7))
        n=5
        ax = fig.add_subplot(1, 2, 1, projection='3d')
        x=y=linspace(-1.,1.,100)
        x,y=meshgrid(x,y)
        z=sin(pi*(n+1)*x)**2/sin(pi*x)**2*sin(pi*(n+1)*y)**2/sin(pi*y)**2
        graphe=ax.plot_surface(x, y, z, rstride=1, cstride=5, cmap=cm.coolwarm)

        ax = fig.add_subplot(1, 2, 2, projection='3d')
        graphe=ax.plot_wireframe(x, y, z, rstride=1, cstride=5, linewidth=1,)
```



B.2 Numpy

Numpy offers a collection of essential tools for scientific computation, such as multidimensional tables, sophisticated functions and much more.

Note that you can use this library to easily integrate C/C++ code or FORTRAN.

B.3 Scipy

This library provides many very efficient digital routines, especially for digital integration and optimization.

In addition, if you need physical constants, you can call them using the following command :

`In [1] : from scipy.constants import *`
and for instance if you enter `In [2] hbar` the ressult is
`Out [2] : 1.0545717253362894e-34`
i.e. the value of Planck's constant divided by 2π

B.4 Other libraries

There are many other bookstores, which you are invited to explore, to name a few :
`Sympy`, for symbolic calculation.
In addition, this library allows to generate LaTeXcode, but requires the use of the `amsmath` package in LaTeX.

`Panda` for data analysis.

For dynamic systems `PyDSTool`. This library requires `numpy`, `scipy` et `matplotlib`

There is a suite, called `SAGE`, which is very suitable for manipulating algebra and geometry.
A number of libraries allow you to interface the input-output ports of your computer or acquisition cards such as `PyVISA` :(GPIB, RS232, and USB ports), `PyUSB` (USB port), `pySerial` (RS232 ports), and pyParallel to manage parallel ports.

Also note the `PIL` library which is extremely useful if you have to manipulate and process images.

B.5 Softwares

In addition to libraries, some software integrates the use of Python. Thus there exist a very advanced free software, allowing the images synthesis and animations. It has a PYTHON console, and all its functions can be managed by PYTHON : BLENDER™ (https ://www.blender.org/). For your leisure ...

Annexe C

Références

PYTHON : https ://www.python.org

Anaconda : https ://www.anaconda.com

Jupyter : https ://jupyter.org

IPYTHON notebooks : https ://ipython.org/ipython-doc/3/notebook/index.html

LaTeX : https ://www.latex-project.org/

CoCal : https ://cocalc.com